Application de recettes en React et TypeScript

Module	Développement d’interfaces avec un framework JavaScript
Objectif	Développer une application React et TypeScript à partir de maquettes fonctionnelles et d’une API REST publique
Durée	3 heures 30
Compétences évaluées	Développer une interface avec un framework JavaScript · Créer des composants réutilisables · Gérer l’état d’une application · Implémenter une navigation entre pages · Typer une application front-end
Documents autorisés	Tous documents et accès internet
Matériel	Ordinateur personnel avec accès internet, Node.js 24+, gestionnaire de paquets au choix (npm, yarn ou pnpm), éditeur de code, navigateur
Technologies imposées	React 19 (18 minimum), TypeScript, Vite, React Router
Prérequis	Aucun : le projet est créé de zéro pendant la séance
Modalité	Individuel

⚠️ CONSIGNES DE RENDU

Format du rendu : le lien d'un dépôt Git (GitHub ou GitLab) accessible au correcteur. Seul le dernier commit poussé avant la fin de la séance est corrigé.

- Le dépôt contient `package.json`, le fichier de verrouillage du gestionnaire de paquets réellement utilisé (`package-lock.json`, `yarn.lock` ou `pnpm-lock.yaml`, un seul) et le fichier `.env` ; ce dernier ne contenant aucun secret, il est versionné pour que le projet démarre chez le correcteur.
- Le dossier `node_modules` est exclu par le `.gitignore`.

Projet exécutable : le correcteur doit pouvoir cloner le dépôt, installer les dépendances puis lancer le serveur de développement, sans étape supplémentaire ni correction manuelle. Un projet qui ne démarre pas perd l'intégralité des points liés au fonctionnement.

UTILISATION DE L'IA GÉNÉRATIVE

L'utilisation d'outils d'IA générative (ChatGPT, Claude, Copilot, Gemini, etc.) est autorisée pendant cette séance. Cependant, elle doit être systématiquement **transparente et justifiée**. La transparence sur l'usage de l'IA fait partie intégrante de la notation.

Règles à respecter impérativement

- Si vous utilisez l'IA pour vous aider (déboguer, générer un composant, rappeler la signature d'un hook, rédiger un paragraphe, etc.), vous devez le **signaler explicitement dans votre code à l'endroit concerné**.
- Pour chaque utilisation, précisez : le **fournisseur et le modèle** utilisés (ex. : ChatGPT GPT-4o, Claude Sonnet, GitHub Copilot…), le **prompt exact** que vous avez soumis, et la **réponse obtenue** (ou un résumé si elle est longue).
- Expliquez **pourquoi** vous avez choisi d'utiliser l'IA plutôt qu'une recherche classique (documentation officielle, cours). Exemples de justifications acceptées : gain de temps sur un problème bloquant, reformulation d'un concept mal compris, génération d'un squelette de composant à adapter.
- Indiquez si vous avez **modifié ou adapté** la réponse de l'IA, et si oui, en quoi et pourquoi.

Ce qui sera valorisé

- Une utilisation pertinente et ciblée de l'IA (pas du copier-coller intégral sans recul)
- La transparence totale : mieux vaut déclarer une utilisation que de la masquer
- La capacité à expliquer et critiquer la réponse de l'IA (montrer que vous comprenez)

Ce qui sera pénalisé

- L'utilisation non déclarée d'IA détectée lors de la correction
- Le copier-coller brut d'une réponse IA sans adaptation, sans explication et sans recul critique
- L'incapacité à expliquer ou défendre un élément de votre rendu produit par IA
- L'apparition dans le `package.json` de dépendances que le sujet interdit, symptôme d'une génération non relue

Conseil : vous pouvez dédier une sous-section « Utilisation de l'IA » dans le README, ou annoter directement chaque passage concerné. L'important est que le correcteur puisse identifier clairement ce qui vient de vous et ce qui vient d'un outil.

Objectif

Développer, à partir des maquettes fournies, l'application web de recettes de la start-up MIJOTÉ en React et TypeScript : catalogue avec recherche, filtre et tri, fiche recette, liste de courses globale, authentification et formulaire de proposition de recette.

Le sujet est rédigé **dans l'ordre d'exécution** : chaque partie s'appuie sur ce que la précédente a mis en place. Lisez d'abord le contexte, les maquettes et l'API, puis traitez les parties dans l'ordre. Chaque exigence renvoie à l'écran concerné (A à G).

Répartition indicative du temps

Partie	Durée indicative
Lecture : contexte, maquettes et API	~10 min
Partie 1 : Initialisation, types et service API	~30 min
Partie 2 : Routage, catalogue et fiche recette	~60 min
Partie 3 : Liste de courses et authentification	~50 min
Partie 4 : Formulaire « Proposer une recette »	~35 min
Partie 5 : Performance, robustesse et livrables	~25 min
TOTAL	**3 h 30**

Contexte général

MIJOTÉ est une jeune start-up française qui veut aider les étudiants et les jeunes actifs à cuisiner plus souvent chez eux. Son idée : trouver une recette en quelques secondes, puis générer d'un clic la liste de courses correspondante.

L'équipe back-end n'a pas terminé son API. En attendant, le lead developer a retenu l'**API publique DummyJSON**, dont le jeu de données « recettes » a **exactement le même contrat d'interface** que la future API de production. Vous intervenez en tant que **développeur front-end** et êtes chargé de réaliser la première version de l'application, conformément aux maquettes validées par le client.

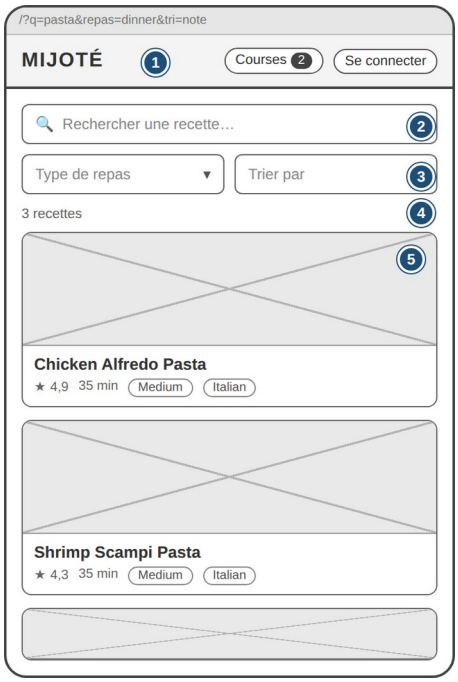
Contraintes communiquées par le lead developer

- **Le mobile reste prioritaire** : 80 % des utilisateurs cuisinent avec leur téléphone posé sur le plan de travail.
- **Le contenu des recettes reste en anglais**, tel que fourni par l'API ; seule l'interface est en français.

Maquettes fonctionnelles (wireframes)

Les maquettes ci-dessous sont des **fils de fer** : elles fixent la structure, les contenus et les comportements attendus, **pas le graphisme**, qui reste libre. Les pastilles numérotées renvoient aux légendes. La colonne de droite indique la partie du sujet où l'écran est développé.

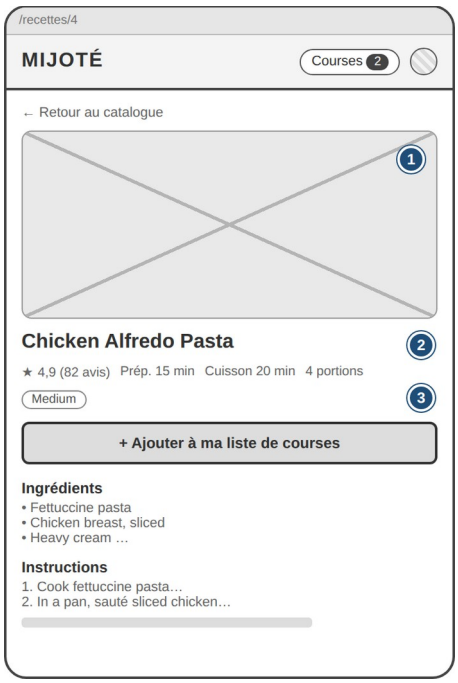
Les six écrans mobiles (360 px) sont présentés dans l'ordre de navigation d'un utilisateur, suivis d'une vue bureau du catalogue.



Écran A : Catalogue

Route `/` · Parties 2.2 et 2.3

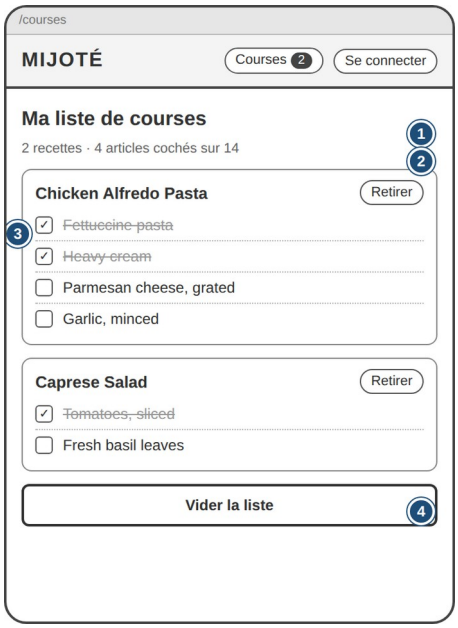
1. **En-tête commun** à toutes les pages : logo (lien vers le catalogue), lien « Courses » avec le **nombre de recettes** dans la liste, bouton « Se connecter » ou avatar et « Se déconnecter ».
2. **Recherche** par nom de recette.
3. **Filtre** par type de repas et **tri**.
4. **Nombre de résultats** affichés.
5. **Carte recette** : image, nom, note, temps total (préparation + cuisson), difficulté, cuisine. Toute la carte mène à la fiche.



Écran B : Fiche recette

Route `/recettes/:id` · Parties 2.4 et 3.1

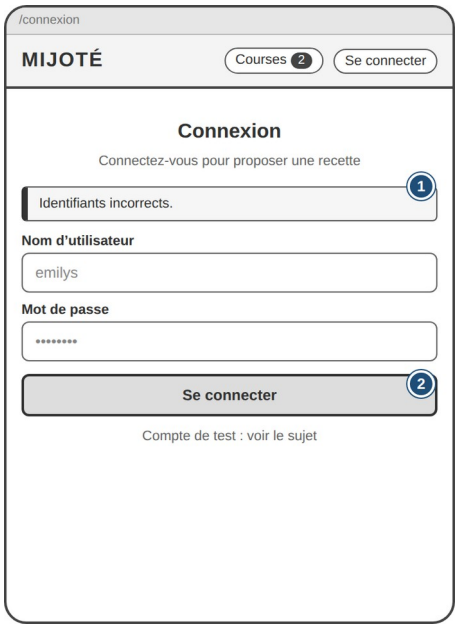
1. **Image** de la recette.
2. **Informations** : note et nombre d'avis, temps de préparation, temps de cuisson, portions, difficulté.
3. **Ajout à la liste de courses** : ajoute tous les ingrédients de la recette. Si la recette est déjà dans la liste, le bouton devient « Déjà dans ma liste » et se désactive.
4. Sous le bouton : ingrédients (liste à puces) puis instructions (liste numérotée).



Écran C : Liste de courses

Route `/courses` · Partie 3.1

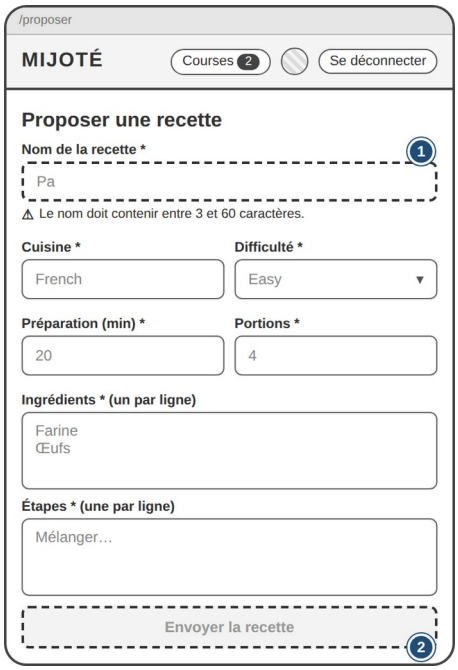
1. **Résumé** : nombre de recettes et d'ingrédients cochés sur le total.
2. **Un groupe par recette**, avec un bouton « Retirer » qui supprime la recette et ses ingrédients.
3. **Case à cocher** par ingrédient ; l'ingrédient coché est barré.
4. **Vider la liste**. Liste vide : message explicite et lien vers le catalogue.



Écran D : Connexion

Route `/connexion` · Parties 3.2 et 3.3

1. **Message d'erreur générique** en cas d'identifiants incorrects : il ne dit pas lequel des deux champs est faux.
2. Bouton de connexion.



Écran E : Proposer une recette

Route `/proposer (protégée)` · Parties 3.3 et 4

1. **Champ fautif** et son message d’erreur.
2. **Bouton d’envoi**, ici désactivé car le formulaire est invalide.



Écran F : Page 404 et états

Route `* et /recettes/9999` · Parties 2.4 et 5

1. **Page 404** soignée pour toute route inconnue, et message « Recette introuvable » pour un identifiant de recette inconnu.
2. **Trois états** à prévoir sur tout écran qui interroge l’API : chargement, erreur (avec bouton « Réessayer »), aucun résultat.

Écran G : Catalogue, vue bureau (≥ 1024 px)



- En bureau, l’en-tête affiche aussi les liens « Catalogue » et « Proposer une recette » et le prénom de l’utilisateur connecté.
- La grille passe de 1 colonne (mobile) à 2 colonnes (tablette, dès 600 px) puis 4 colonnes (bureau, dès 1024 px). Aucun défilement horizontal n’apparaît, quelle que soit la largeur.

L’API de recette

URL de base : `https://dummyjson.com` (documentation : `https://dummyjson.com/docs/recipes` et `https://dummyjson.com/docs/auth`). L’API est gratuite, ne demande aucune clé et accepte les appels depuis le navigateur.

Méthode	Endpoint	Description
`GET`	`/recipes/search?q={texte}&limit=0`	Recherche par nom. Avec `q` vide, renvoie les 50 recettes.
`GET`	`/recipes/{id}`	Fiche d’une recette. Répond **404** si l’identifiant n’existe pas.
`POST`	`/auth/login`	Connexion : renvoie l’utilisateur et un jeton.
`POST`	`/recipes/add`	Création d’une recette (**simulée**).
`GET`	`/auth/me`	Utilisateur courant, jeton dans l’en-tête `Authorization: Bearer {jeton}`. Utilisé uniquement pour un point bonus.

> **Attention à la pagination :** sans `limit=0`, l’API ne renvoie que les **30 premiers** résultats (`"limit": 30`). Ajoutez toujours `limit=0` à la recherche.

Réponse de GET /recipes/search?q=pasta&limit=0 (extrait)

```
{
  "recipes": [
    { "id": 4, "name": "Chicken Alfredo Pasta", ... },
    { "id": 10, "name": "Shrimp Scampi Pasta", ... },
    { "id": 46, "name": "Pesto Pasta with Cherry Tomatoes", ... }
  ],
  "total": 3,
  "skip": 0,
  "limit": 3
}
```

Réponse de GET /recipes/1

```
{
  "id": 1,
  "name": "Classic Margherita Pizza",
  "ingredients": ["Pizza dough", "Tomato sauce", "Fresh mozzarella cheese", ...],
  "instructions": ["Preheat the oven to 475°F (245°C).", ...],
  "prepTimeMinutes": 20,
  "cookTimeMinutes": 15,
  "servings": 4,
  "difficulty": "Easy",
  "cuisine": "Italian",
  "caloriesPerServing": 300,
  "tags": ["Pizza", "Italian"],
  "userId": 166,
  "image": "https://cdn.dummyjson.com/recipe-images/1.webp",
  "rating": 4.6,
  "reviewCount": 98,
  "mealType": ["Dinner"]
}
```

Connexion : POST /auth/login

Compte de test : nom d'utilisateur `emilys`, mot de passe `emilyspass`.

```
// Requête (en-tête Content-Type: application/json)
{ "username": "emilys", "password": "emilyspass" }

// Réponse 200
{
  "id": 1,
  "username": "emilys",
  "firstName": "Emily",
  "image": "https://dummyjson.com/icon/emilys/128",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  ...
}

// Réponse 400 (identifiants incorrects)
{ "message": "Invalid credentials" }
```

Charge utile attendue par POST /recipes/add

Attention : l'API **simule** la création de la recette. Elle répond avec un identifiant (51) mais n'enregistre rien : la recette n'apparaîtra jamais dans le catalogue. Traitez la réponse comme un succès sans chercher à la relire ni à l'injecter dans la liste.

```
{
  "name": "Crêpes au sucre",
  "cuisine": "French",
  "difficulty": "Easy",
  "prepTimeMinutes": 20,
  "servings": 4,
  "ingredients": ["Farine", "Œufs", "Lait", "Sucre"],
  "instructions": ["Mélanger la farine et les œufs.", "Ajouter le lait."],
  "userId": 1
}

// Réponse : la recette envoyée, complétée d'un identifiant
{ "id": 51, "name": "Crêpes au sucre", ... }
```

Partie 1 : Initialisation, types et service API

1.1 Création du projet

- Le projet est créé avec Vite, template **React + TypeScript** (`react-ts`).
- Dépendances : React, React Router (paquet `react-router`) et l'outillage TypeScript/Vite. **Aucune bibliothèque de composants, de formulaires ni de framework CSS** (ni MUI, Chakra, React Hook Form, Formik, Bootstrap ou Tailwind) : le style est écrit à la main. Toute autre dépendance est justifiée dans le README.
- Un fichier `.env` à la racine contient `VITE_API_BASE_URL=https://dummyjson.com`, seule source de l'URL de l'API. Elle est lue via `import.meta.env` et son type est déclaré dans l'interface `ImportMetaEnv` d'un fichier de déclaration `.d.ts`.
- Le mode `strict` de TypeScript est actif. La commande de build de production s'exécute **sans erreur ni avertissement TypeScript**.

1.2 Types du domaine

Déclarez les types une seule fois dans `src/types/` et réutilisez-les partout. `any` n'apparaît pas, sauf commentaire le justifiant.

Type	Contenu
`Recipe`	Une recette, avec au minimum les champs affichés sur les écrans A et B
`RecipeSearchResponse`	La réponse de la recherche : `recipes`, `total`, `skip`, `limit`
`MealType`	Union des six valeurs du filtre : `'breakfast' | 'lunch' | ...`
`Difficulty`	`'Easy' | 'Medium' | 'Hard'`
`LoginCredentials`, `AuthUser`	Les données envoyées et reçues par `/auth/login`
`NewRecipe`	La charge utile envoyée à `/recipes/add`
`ShoppingListEntry`	Une recette de la liste de courses : `recipeId`, `recipeName`, `items: { label: string; checked: boolean }[]`

1.3 Service API

- `src/services/api.ts` expose `searchRecipes(query, signal?)`, `getRecipeById(id, signal?)`, `login(credentials)` et `createRecipe(payload)`, construites sur `fetch` et lisant l'URL de base depuis la variable d'environnement.
- Chaque fonction vérifie `response.ok`. En cas d'échec, elle lève une erreur qui **conserve le code HTTP** (par exemple une classe `ApiError` avec un champ `status`), pour que l'interface distingue un 404 d'une panne réseau.
- Les réponses sont typées ; aucune valeur reçue n'est manipulée en `any`.

1.4 Hook de requête

- Un hook générique `useFetch<T>` expose les trois états d'une requête : `{ data, loading, error }`, ainsi qu'une fonction `reload` pour le bouton « Réessayer ».
- Il annule la requête en cours avec un `AbortController` quand ses paramètres changent ou quand le composant est démonté. Une annulation n'est pas affichée comme une erreur.
- Les tableaux de dépendances des `useEffect` sont exacts : aucune requête ne part en boucle, aucune donnée affichée n'est périmée.

Partie 2 : Routage, catalogue et fiche recette

2.1 Découpage en composants

L'arborescence de `src/` sépare au minimum les composants, les pages, les services, les hooks, les contextes, les types et les styles :

```
src/
├── components/   // Layout, Header, SearchBar, MealTypeFilter, SortSelect,
│                 // RecipeCard, RecipeList, Loader, ErrorMessage, EmptyState,
│                 // ProtectedRoute
├── pages/        // CatalogPage, RecipePage, ShoppingListPage, LoginPage,
│                 // NewRecipePage, NotFoundPage
├── hooks/        // useFetch, useDebounce, useLocalStorage
├── context/      // ShoppingListContext, AuthContext
├── services/     // api.ts
├── types/        // Recipe, MealType, AuthUser, NewRecipe, ShoppingListEntry...
├── styles/
├── App.tsx
└── main.tsx
```

- Chaque composant a une responsabilité unique et reçoit par *props* typées ce dont il a besoin.
- Au moins un composant est effectivement réutilisé dans deux contextes différents (par exemple `Loader` et `ErrorMessage` sur les écrans A et B).

2.2 Routage et gabarit commun

Route	Écran
`/`	A : Catalogue
`/recettes/:id`	B : Fiche recette
`/courses`	C : Liste de courses
`/connexion`	D : Connexion
`/proposer`	E : Proposer une recette (protégée en partie 3.3 ; accessible à tous en attendant)
`*`	F : Page 404 soignée

- Un composant `Layout` affiche l'en-tête puis la page courante via `<Outlet />`. À ce stade, l'en-tête contient le logo et les liens ; le compteur de la liste et le bouton de connexion sont branchés en partie 3.
- Le lien de la page active est signalé visuellement et par `aria-current="page"` (`NavLink` le gère).

2.3 Catalogue : recherche, filtre et tri (écrans A et G)

- Au premier affichage, le catalogue appelle `searchRecipes('')`.
- La **recherche** est un champ contrôlé. Un hook `useDebounce` retarde l'appel de 400 ms après la dernière frappe.

- Le **filtre par type de repas** (six valeurs, voir le tableau ci-dessous) et le **tri** s'appliquent en mémoire sur la liste reçue, **sans nouvel appel**. Options de tri : Pertinence (ordre de l'API), Note décroissante, Temps total croissant, Nom de A à Z.

Valeurs du filtre. L'API ne fournit pas la liste des types de repas : utilisez les six valeurs ci-dessous. Dans les données, elles sont écrites avec une majuscule (`"Dinner"`) : la comparaison doit **ignorer la casse**.

Valeur (URL)	Valeur dans les données	Libellé affiché
`breakfast`	`Breakfast`	Petit-déjeuner
`lunch`	`Lunch`	Déjeuner
`dinner`	`Dinner`	Dîner
`appetizer`	`Appetizer`	Entrée
`dessert`	`Dessert`	Dessert
`beverage`	`Beverage`	Boisson

- La barre de recherche, le filtre et le tri sont **trois composants distincts** qui agissent sur une même liste ; leur état vit dans la page.
- La liste filtrée et triée est mémoïsée par `useMemo`. Le tri ne modifie pas le tableau d'origine.
- Les trois états de l'écran F sont gérés par les composants `Loader`, `ErrorMessage` et `EmptyState`.
- La recherche, le filtre et le tri sont synchronisés avec l'URL via `useSearchParams` (`/?q=pasta&repas=dinner&tri=note`), de sorte qu'une recherche filtrée puisse être partagée par lien et survive à un rechargement. Point bonus, non comptabilisé dans la notation.

2.4 Fiche recette (écrans B et F)

- `useParams` lit l'identifiant, puis `getRecipeById` charge la recette.
- Un identifiant inconnu (réponse 404) affiche le message « Recette introuvable » de l'écran F, **distinct** de celui d'une erreur réseau.
- La fiche suit l'écran B : l'image a un texte alternatif, les ingrédients sont dans une liste `<ul>` et les instructions dans une liste `<ol>`.

Partie 3 : Liste de courses et authentification

3.1 Liste de courses : état global (écrans B et C)

La liste de courses est lue et modifiée depuis trois endroits : la fiche, la page `/courses` et l'en-tête. C'est donc un état global.

- Un `ShoppingListContext` alimenté par `useReducer` porte un tableau de `ShoppingListEntry`. Les actions sont typées par une **union discriminée** et couvrent au minimum :
  - `ADD_RECIPE` : ajoute une recette et tous ses ingrédients, non cochés. Une recette déjà présente n'est pas ajoutée une seconde fois.
  - `TOGGLE_ITEM` : coche ou décoche un ingrédient.
  - `REMOVE_RECIPE` : retire une recette et ses ingrédients.
  - `CLEAR` : vide la liste.
- Le *reducer* est une fonction pure : il ne mute jamais l'état reçu.
- La liste est persistée dans `localStorage` et restaurée au chargement. Une valeur absente ou illisible (JSON invalide) donne une liste vide, sans plantage.
- Le bouton de la fiche (écran B, repère 3) et la page `/courses` (écran C, état vide compris) sont branchés sur ce contexte.
- Cocher un ingrédient **ne doit pas re-rendre l'en-tête**, qui ne consomme que le nombre de recettes : découpez le contexte (par exemple état et actions séparés, ou un contexte dédié au compteur) et vérifiez-le au React Profiler.

3.2 Authentification (écran D)

- Un `AuthContext` expose l'utilisateur connecté (`AuthUser | null`), le jeton et les fonctions `login` et `logout`.
- La page `/connexion` envoie `POST /auth/login` et traite trois cas : envoi en cours (bouton désactivé), identifiants incorrects (réponse 400, message de l'écran D), panne réseau (message distinct).
- L'en-tête bascule selon l'état de connexion (écrans A et G) ; l'avatar a un texte alternatif.
- Le mode de stockage du jeton est explicite, `localStorage` ou mémoire, et cohérent avec le comportement attendu au rechargement ; une ligne du README indique lequel et pourquoi.
- Si un jeton est stocké, l'application le vérifie au démarrage avec `GET /auth/me` (en-tête `Authorization: Bearer {jeton}`) et déconnecte l'utilisateur si la réponse n'est pas `ok`. Point bonus, non comptabilisé dans la notation.

3.3 Route protégée

- Un composant `ProtectedRoute` protège `/proposer`. Un visiteur non connecté est redirigé vers `/connexion` avec `<Navigate replace />`, en transmettant la page demandée dans l'état de navigation (`state`).
- Après connexion, `useNavigate` ramène l'utilisateur sur la page demandée, ou sur le catalogue s'il n'en avait pas.
- Après déconnexion depuis `/proposer`, l'utilisateur est redirigé vers le catalogue.

Partie 4 : Formulaire « Proposer une recette » (écran E)

4.1 Champs et règles de validation

Champ	Règle
Nom de la recette	Obligatoire, 3 à 60 caractères une fois les espaces de début et de fin retirés
Cuisine	Obligatoire, non vide
Difficulté	Liste déroulante : `Easy`, `Medium` ou `Hard`
Préparation (min)	Nombre entier de 1 à 240
Portions	Nombre entier de 1 à 12
Ingrédients	Zone de texte, un ingrédient par ligne, au moins 2 lignes non vides
Étapes	Zone de texte, une étape par ligne, au moins 1 ligne non vide

- La validation est écrite dans une **fonction pure** `validate(values)` qui renvoie un objet d'erreurs typé, sans dépendre de React.

4.2 Comportement et accessibilité

- L'erreur d'un champ s'affiche après sa première perte de focus ou à la tentative d'envoi, et non dès le premier caractère saisi.
- Le bouton d'envoi reste désactivé tant que le formulaire est invalide.
- Chaque champ a un `<label>` associé. Un champ fautif porte un retour visuel **et** un message d'erreur perceptible autrement que par la couleur, associé au champ par `aria-describedby` et `aria-invalid`.

4.3 Envoi

- À la soumission, les zones de texte sont découpées en tableaux (lignes vides retirées), puis `POST /recipes/add` est envoyé avec la charge utile `NewRecipe` décrite dans la partie API. Le champ `userId` vient de l'utilisateur connecté.
- L'interface traite trois états : envoi en cours (bouton désactivé, libellé « Envoi… »), succès (message indiquant l'identifiant renvoyé, formulaire réinitialisé), échec (message d'erreur, saisie conservée).

Partie 5 : Performance, robustesse et livrables

5.1 Performance et robustesse

- **Un** re-rendu inutile réellement observé au React Profiler est corrigé avec `React.memo`, `useMemo` ou `useCallback` (par exemple les cartes du catalogue qui se re-rendent à chaque frappe) ; le README indique le composant concerné et ce qui a été mesuré avant et après.
- L'application est utilisable au clavier : ordre de tabulation logique, focus visible, cartes et boutons activables avec Entrée.
- Les routes sont chargées à la demande via `React.lazy` et `Suspense`, et l'effet sur la taille des fichiers produits par le build est mesuré. Point bonus, non comptabilisé dans la notation.

5.2 Livrables

En plus du code, le dépôt Git contient :

- Un `README.md` : présentation, prérequis, gestionnaire de paquets utilisé et commandes exactes, scripts disponibles, variables d'environnement, arborescence, limites connues, ainsi que les éléments demandés en parties 1.1, 3.2 et 5.1 et la section « Utilisation de l'IA générative ».
- Un schéma ou une arborescence commentée, placé dans le README, montrant comment les données circulent des contextes vers les composants de présentation.
- Les captures d'écran des six écrans mobiles (A à F) et de la vue bureau (G), placées dans un dossier `docs/`.

Annexe : Ressources autorisées

- **Documentation React :** `https://react.dev`
- **React Router :** `https://reactrouter.com`
- **Vite :** `https://vite.dev`
- **MDN Web Docs :** `https://developer.mozilla.org`
- **DummyJSON, recettes :** `https://dummyjson.com/docs/recipes`
- **DummyJSON, authentification :** `https://dummyjson.com/docs/auth`
- Outils conseillés : VS Code (Prettier, ESLint), React Developer Tools, Chrome DevTools

Fin du sujet